

THE MATCOM MINIX

Informe Técnico

Estudiante: Eric Reyes Milian

Grupo: 212

8 de mayo de 2026

Índice

1. Introducción	2
2. Desarrollo del proyecto	2
2.1. Instalación y configuración de VirtualBox y MINIX	2
2.2. Personalización del mensaje de bienvenida	2
2.3. Depuración de un bug en <code>pthread</code>	3
2.4. Implementación del comando <code>tree</code>	4
2.5. Modificación del scheduler: penalización de procesos CPU-bound	6
2.5.1. Análisis previo	6
2.5.2. Diseño de la política	6
2.5.3. Implementación	7
2.5.4. Validación experimental	7
3. Discusión de resultados	7
4. Conclusiones	7
5. Referencias consultadas	8
6. Uso de herramientas de IA	8
7. Anexos	8

1. Introducción

La asignatura Sistemas Operativos aborda principios fundamentales relacionados con la gestión de procesos, planificación de CPU, administración de memoria y organización del sistema de archivos. En este proyecto se utilizó MINIX como plataforma de estudio debido a su arquitectura modular basada en microkernel, la cual permite analizar y modificar componentes reales de un sistema operativo de manera práctica.

El objetivo principal del trabajo fue consolidar los conocimientos adquiridos durante el curso mediante la implementación, depuración y modificación de distintos subsistemas de MINIX. Para ello, se realizaron tareas orientadas a comprender el funcionamiento interno del sistema operativo y experimentar directamente con su código fuente.

Entre las actividades desarrolladas se encuentran la personalización del mensaje de bienvenida del sistema, la corrección de un bug relacionado con `pthread_mutex_trylock`, la implementación del comando `tree` para visualizar estructuras de directorios y la modificación del scheduler para incorporar un mecanismo de penalización de procesos CPU-bound basado en el consumo sostenido de CPU.

2. Desarrollo del proyecto

2.1. Instalación y configuración de VirtualBox y MINIX

Como primera etapa del proyecto se realizó la instalación y configuración del entorno de trabajo utilizando Oracle VirtualBox y la última versión estable de MINIX. Para garantizar un funcionamiento estable del sistema y evitar problemas durante la compilación y clonación del repositorio, se configuró la máquina virtual con una cantidad suficiente de memoria RAM pre-asignada, evitando así errores relacionados con falta de memoria durante operaciones intensivas de entrada/salida.

Posteriormente se instalaron los paquetes necesarios para el desarrollo utilizando `pkgin_sets`, lo que permitió disponer de herramientas esenciales como compiladores, utilidades de desarrollo y soporte para control de versiones. Una vez preparado el entorno, se configuró Git estableciendo los datos básicos del usuario y habilitando el acceso al repositorio del proyecto. Debido a limitaciones de certificados SSL presentes en la instalación inicial de MINIX, la clonación del repositorio se realizó deshabilitando temporalmente la verificación SSL del protocolo HTTPS.

Además, se configuró el intercambio de archivos entre el sistema anfitrión y MINIX mediante el montaje de carpetas compartidas de VirtualBox. Inicialmente el montaje se realizó manualmente para verificar su correcto funcionamiento. Posteriormente, con el objetivo de automatizar el proceso y facilitar el flujo de trabajo, se modificó el archivo `/etc/rc` para ejecutar automáticamente el montaje durante el arranque del sistema operativo. De esta manera, la carpeta compartida quedaba disponible en cada inicio de MINIX sin intervención manual adicional. Esto permitió establecer un canal de intercambio eficiente entre el sistema anfitrión y MINIX.

La estrategia utilizada durante el desarrollo consistió en clonar el repositorio de MINIX en el sistema anfitrión, aplicar los cambios en un entorno más cómodo y posteriormente copiar los archivos modificados hacia la carpeta compartida para reemplazarlos en sus ubicaciones correspondientes dentro del sistema operativo.

2.2. Personalización del mensaje de bienvenida

Ruta modificada: `/etc/motd` (repositorio: `etc/motd`).

El primer hito introdujo una personalización explícita del mensaje de bienvenida con referencia a la Facultad de Matemática y Computación de la Universidad de La Habana.

Inicialmente la modificación se realizó manualmente desde MINIX utilizando el editor de texto `mined`. Sin embargo, debido a limitaciones relacionadas con edición y manipulación de

texto, se optó posteriormente por reemplazar el archivo `/etc/motd` mediante una copia desde la carpeta compartida utilizando:

```
cp -f [archivo_origen] [archivo_destino]
```

La opción `-f` permitió forzar el reemplazo del contenido original.

```
1          #####
2          #####
3          +#####
4          -#####+
5          #####
6          #####
7          #####
8          #####
9          #####          #####
10         #####          #####
11         -#####          +#####
12         +#####          #####
13         ######          ###
14
15             MINIX 3
16             EricReyesMilian
```

Listing 1: Contenido de `etc/motd`

2.3. Depuración de un bug en pthread

Síntoma observado: la segunda invocación a `pthread_mutex_trylock` provocaba que el programa quedara bloqueado indefinidamente.

En MINIX, las funciones `pthread_*` funcionan como una capa de compatibilidad sobre las implementaciones reales `pthread_*`. Revisando el fichero `minix/lib/libmthread/pthread_compat.c`, se detectó que la implementación de `pthread_mutex_trylock` realizaba una llamada recursiva a sí misma en lugar de delegar correctamente en `mthread_mutex_trylock`.

```
1 int pthread_mutex_trylock(pthread_mutex_t *mutex)
2 {
3     if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
4         mthread_mutex_init(mutex, NULL);
5     }
6
7     return pthread_mutex_trylock(mutex);
8 }
```

Listing 2: Implementación errónea

La solución consistió en reemplazar la llamada recursiva por la función correcta:

```
1 int pthread_mutex_trylock(pthread_mutex_t *mutex)
2 {
3     if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
4         mthread_mutex_init(mutex, NULL);
5     }
6
7     return mthread_mutex_trylock(mutex);
8 }
```

Listing 3: Implementación corregida

Luego de corregido el problema principal, surgió una situación interesante durante la validación experimental. Aunque el comportamiento era correcto, el valor numérico asociado a `EDEADLK` no coincidía con el esperado según la guía del proyecto.

La salida esperada era:

```
first trylock: 0 (OK)
second trylock: 35 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS
```

Sin embargo, en MINIX se obtuvo:

```
first trylock: 0 (OK)
second trylock: 11 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS
```

Esto motivó una investigación adicional sobre los valores internos de `errno.h` en distintos sistemas operativos.

```
1 #include <stdio.h>
2 #include <errno.h>
3
4 int main() {
5     printf("El valor de EDEADLK en este sistema es: %d\n",
6           EDEADLK);
7     return 0;
8 }
```

Listing 4: Programa utilizado para verificar EDEADLK

Resultados obtenidos:

- MINIX: 11
- Linux: 35
- Windows: 36

A partir de esta validación se concluyó que la implementación era correcta y que la diferencia correspondía únicamente a la representación numérica interna del código de error.

2.4. Implementación del comando `tree`

La implementación del comando `tree` se integró como un nuevo comando de usuario mediante la creación de los archivos:

- `bin/tree/tree.c`
- `bin/tree/Makefile`
- inclusión de `tree` dentro de `bin/Makefile`

El algoritmo fue diseñado utilizando un enfoque recursivo explícito mediante la función:

```
print_tree(path, depth, total_dirs, total_files)
```

La recursividad permitió representar naturalmente la estructura jerárquica del sistema de archivos, utilizando el parámetro `depth` para controlar visualmente la indentación.

Para el recorrido de directorios se utilizaron las llamadas al sistema:

- `opendir()`
- `readdir()`
- `closedir()`
- `lstat()`

La función `lstat()` permitió obtener metadatos sin seguir enlaces simbólicos, evitando ciclos infinitos.

```
1 if (lstat(full_path, &st) != 0) {
2     free(entries[i]);
3     continue;
4 }
5
6 if (S_ISDIR(st.st_mode)) {
7
8     if (!S_ISLNK(st.st_mode)) {
9         printf("|-- %s/\n", entry->d_name);
10        (*total_dirs)++;
11
12        print_tree(full_path, depth + 1,
13                    total_dirs, total_files);
14    } else {
15        printf("|-- %s [enlace simbolico]\n",
16                entry->d_name);
17    }
18 }
```

Listing 5: Fragmento principal del recorrido recursivo

También se implementó una función auxiliar denominada `should_skip()` para ignorar las entradas especiales `.` y `..`, evitando recursión infinita.

Ejemplo de estructura procesada:

```
proyecto/
  main.c
  Makefile
  include/
    tree.h
  src/
    tree.c
    utils.c
```

Salida producida por el comando:

```
proyecto
|-- main.c
|-- Makefile
|-- include/
```

```
|    |-- tree.h
|-- src/
|    |-- tree.c
|    |-- utils.c
```

2 directorios, 4 archivos

2.5. Modificación del scheduler: penalización de procesos CPU-bound

Uno de los objetivos principales del proyecto consistió en modificar el scheduler de MINIX para incorporar un mecanismo de penalización dinámica basado en el uso sostenido de CPU. La política implementada busca distinguir entre procesos interactivos y procesos CPU-bound, ajustando temporalmente la prioridad de ejecución según el comportamiento observado durante ventanas periódicas de planificación.

2.5.1. Análisis previo

Antes de realizar modificaciones sobre el scheduler fue necesario estudiar la estructura interna del servicio `sched` y comprender el flujo de planificación de procesos dentro de MINIX.

Se analizaron particularmente las siguientes funciones:

- `do_noquantum()`
- `balance_queues()`
- `do_start_scheduling()`
- `do_stop_scheduling()`

Asimismo, se identificó que la información de planificación de cada proceso se almacena en la estructura `schedproc`, definida en `schedproc.h`.

2.5.2. Diseño de la política

La política implementada se basó en ventanas temporales de planificación aprovechando el mecanismo periódico ya existente en MINIX.

Para cada proceso se mantuvo un contador denominado:

`quantum_count`

encargado de registrar cuántas veces el proceso agotaba completamente su quantum dentro de una ventana de observación.

Parámetros definidos:

- `CPU_BOUND_QUANTUM_THRESHOLD = 3`
- `MAX_PENALTY_LEVEL = 2`

Campos añadidos a `schedproc`:

```
1 unsigned base_priority;
2 unsigned quantum_count;
3 unsigned penalty_level;
```

Listing 6: Campos añadidos a `schedproc`

2.5.3. Implementación

La primera modificación se realizó sobre `do_noquantum()`, donde se incrementa `quantum_count` cuando un proceso consume completamente su quantum.

Posteriormente, en `do_start_scheduling()`, se inicializaron los nuevos campos añadidos a la estructura.

La lógica principal fue incorporada dentro de `balance_queues()`, ejecutada periódicamente cada `BALANCE_TIMEOUT` segundos.

Durante cada ventana:

- Si un proceso agotó al menos tres quantums, se incrementa progresivamente su penalización.
- Si un proceso no agotó ningún quantum, recupera gradualmente prioridad.
- Finalmente se reinicia `quantum_count`.

2.5.4. Validación experimental

La validación se realizó utilizando dos escenarios:

1. Un proceso CPU-bound basado en un bucle infinito.
2. Un proceso interactivo utilizando llamadas a `sleep()`.

Durante las pruebas se observó que el proceso CPU-bound agotaba continuamente sus quantums y era penalizado progresivamente, mientras que el proceso interactivo recuperaba prioridad durante las ventanas donde no consumía CPU intensivamente.

Los resultados obtenidos mostraron un comportamiento coherente con la política diseñada.

3. Discusión de resultados

Todos los componentes desarrollados alcanzaron un estado funcional completo y cumplieron los requisitos establecidos en la guía orientadora. La personalización del mensaje de bienvenida se implementó correctamente, el bug asociado a `pthread_mutex_trylock` fue corregido satisfactoriamente, el comando `tree` permitió representar recursivamente estructuras de directorios utilizando llamadas al sistema, y la modificación del scheduler logró diferenciar procesos CPU-bound de procesos interactivos mediante un mecanismo dinámico de penalización.

Durante el desarrollo surgieron dificultades asociadas principalmente a la comprensión del código interno de MINIX, la depuración de errores de concurrencia y el análisis del comportamiento del scheduler. También fue necesario validar diferencias entre códigos numéricos de error presentes en distintos sistemas operativos.

Las decisiones de diseño adoptadas permitieron mantener soluciones simples, modulares y compatibles con la arquitectura original del sistema.

4. Conclusiones

El desarrollo de este proyecto permitió aplicar de forma práctica los conceptos fundamentales de la asignatura Sistemas Operativos mediante el trabajo directo sobre MINIX y la modificación de componentes reales del sistema. A través de la corrección de un bug en `pthread`, la implementación del comando `tree` y la modificación del scheduler para penalizar procesos CPU-bound, se profundizó en mecanismos relacionados con concurrencia, planificación de procesos y manejo del sistema de archivos. Las pruebas y validaciones realizadas permitieron comprender la importancia de la depuración, el análisis experimental y el diseño de políticas de planificación en sistemas operativos reales.

5. Referencias consultadas

1. Orientación del proyecto.pdf
2. Cronograma de hitos.pdf
3. THE MATCOM MINIX_Informe técnico.pdf
4. Código fuente de MINIX:
 - `etc/motd`
 - `bin/tree/*`
 - `minix/lib/libmthread/pthread_compat.c`
 - `minix/servers/sched/*`

6. Uso de herramientas de IA

Se utilizó IA generativa como apoyo para:

- asistencia en la escritura y estructuración del documento en $\text{\LaTeX}$,
- refactorización y organización de fragmentos de código,
- generación de ejemplos y pruebas auxiliares.

7. Anexos